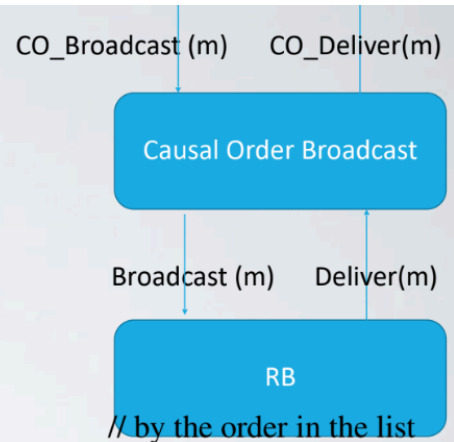


Distributed Systems - 22_10

Let's see the pseudo-code of the algorithm.

```
upon event  $\langle crb, Init \rangle$  do  
     $delivered := \emptyset;$   
     $past := [];$   
  
upon event  $\langle crb, Broadcast \mid m \rangle$  do  
    trigger  $\langle rb, Broadcast \mid [DATA, past, m] \rangle;$   
     $append(past, (self, m));$   
  
upon event  $\langle rb, Deliver \mid p, [DATA, mpast, m] \rangle$  do  
    if  $m \notin delivered$  then  
        forall  $(s, n) \in mpast$  do  
            if  $n \notin delivered$  then  
                trigger  $\langle crb, Deliver \mid s, n \rangle;$   
                 $delivered := delivered \cup \{n\};$   
                if  $(s, n) \notin past$  then  
                     $append(past, (s, n));$   
            trigger  $\langle crb, Deliver \mid p, m \rangle;$   
             $delivered := delivered \cup \{m\};$   
            if  $(p, m) \notin past$  then  
                 $append(past, (p, m));$ 
```



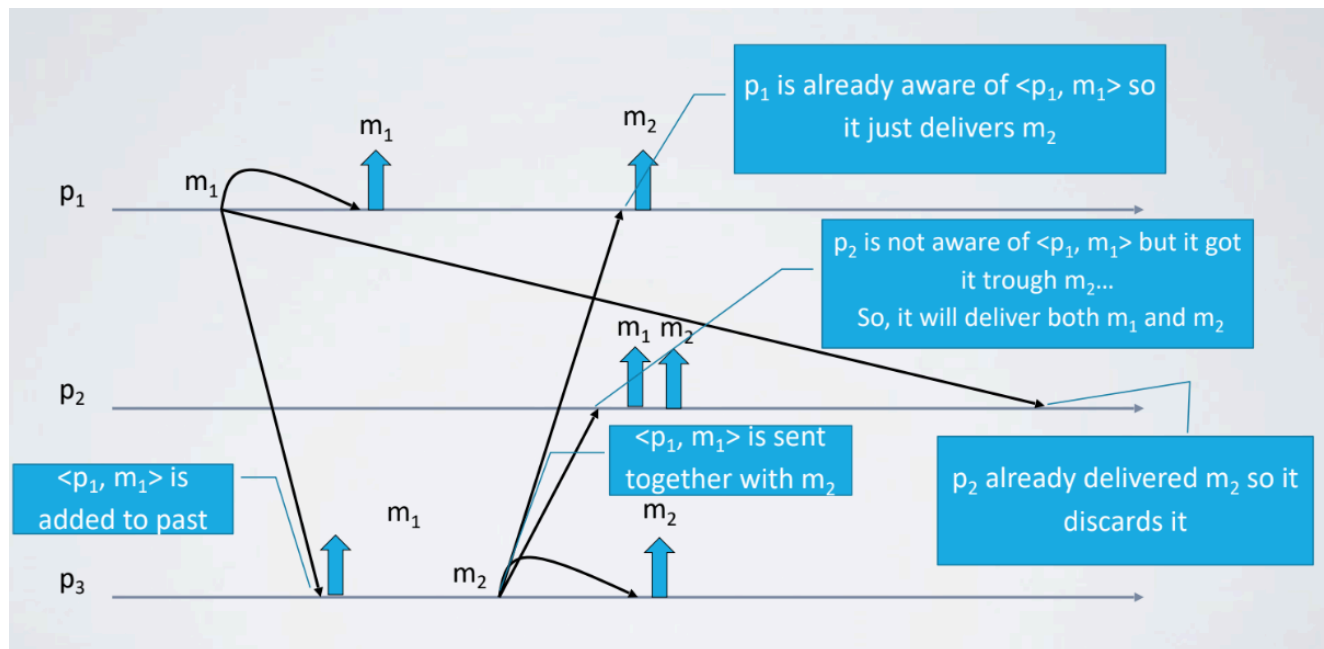
When broadcasting a message, we also append an array `past`. Also, we append to `past` a tuple given by our id and the message.

When the deliver handler triggers in a process, we'll receive the `past` sent by another process. The process will check if it's seeing the message for the first time. Then I check if in `past` there is a message we've not seen (with a for executed by the order in the list), and if so we deliver also the messages in `past`. Only then we can deliver the message which contained the `past`.

This kind of configuration solves the problem seen in the transaction before.

When I receive something, I add it to my past. When I broadcast something, I send my whole history.

Let us see an execution:



At first p_1 will have $[p, [], m]$. Then, when p_3 receives a message his past will be given by $[(p_1, m_1)]$. When sending a message to p_2 , we'll send $[DATA, [(p_1, m_1)], m_2]$. p_2 receives this message before the broadcast of p_1 , and by delivering it immediately we'll have a problem. So we check past, we see there is p_1, m_1 that we have not delivered. So we deliver at first m_1 , and then m_2 . This will mean that the delivered set now contains $\{m_1, m_2\}$. Message m_1 will reach p_2 late, but because of the check of $m \notin \text{delivered}$ we can discard the message.

What is the drawback of this algorithm? We have that the past grows too much. We could fix this thing by deleting something from the past set. We can then remove a message from the past if everyone has received a message, and we'll use ACKs. So, we also have to have a failure detector, or we could wait forever for an ACK coming from a dead process.

So, let's take a look at the improved algorithm (no-wait crb). (the content at right is new, in left no).

<pre> upon event $\langle crb, Broadcast \mid m \rangle$ do trigger $\langle rb, Broadcast \mid [DATA, past, m] \rangle$; $append(past, (self, m))$; upon event $\langle rb, Deliver \mid p, [DATA, mpast, m] \rangle$ do if $m \notin delivered$ then forall $(s, n) \in mpast$ do if $n \notin delivered$ then trigger $\langle crb, Deliver \mid s, n \rangle$; $delivered := delivered \cup \{n\}$; if $(s, n) \notin past$ then $append(past, (s, n))$; trigger $\langle crb, Deliver \mid p, m \rangle$; $delivered := delivered \cup \{m\}$; if $(p, m) \notin past$ then $append(past, (p, m))$; </pre>	<pre> upon event $\langle crb, Init \rangle$ do $delivered := \emptyset$; $past := []$; $correct := \Pi$; forall m do $ack[m] := \emptyset$; upon event $\langle \mathcal{P}, Crash \mid p \rangle$ do $correct := correct \setminus \{p\}$; upon exists $m \in delivered$ such that $self \notin ack[m]$ do $ack[m] := ack[m] \cup \{self\}$; trigger $\langle rb, Broadcast \mid [ACK, m] \rangle$; upon event $\langle rb, Deliver \mid p, [ACK, m] \rangle$ do $ack[m] := ack[m] \cup \{p\}$; upon $correct \subseteq ack[m]$ do forall $(s', m') \in past$ such that $m' = m$ do $remove(past, (s', m'))$; </pre>
--	---

In the init, we'll also have a correct set. We also have the ack set for all the possible messages ever created (as seen in another algorithm).

We also have an handler that says that if $\exists m \in delivered$ such that $self \notin ack[m]$, so basically if we have a message in delivered such that we don't have the ack coming from myself, so we update our set and we broadcast our ACK.

I can remove a message from the past with the last handler, iterating on the past array and removing the messages $m' = m$.

Remember, we have sync systems and Fail Stop systems. If we just have P we cannot reason about time.

So, asking ourselves how much physical time a message can state in the past array is unbounded, that means that the time will be finite, but we don't know how much.

What about how many message delays a message stays in past? Two in the best case, one message delay for the broadcasting, one for the sending of the ACK (ACKs are in parallel).

Even this algorithm is not perfect, if we have a slow processes with slow ACKs, a certain message may remain inside the past set a lot.

How can we improve it? We could use the vector clock.

WAITING CAUSAL RELIABLE BROADCAST

FAIL SILENT

upon event $\langle crb, Init \rangle$ **do**

$V := [0]^N;$

$lsn := 0;$

$pending := \emptyset;$

upon event $\langle crb, Broadcast \mid m \rangle$ **do**

$W := V;$

$W[rank(self)] := lsn;$

$lsn := lsn + 1;$

trigger $\langle rb, Broadcast \mid [DATA, W, m] \rangle;$

upon event $\langle rb, Deliver \mid p, [DATA, W, m] \rangle$ **do**

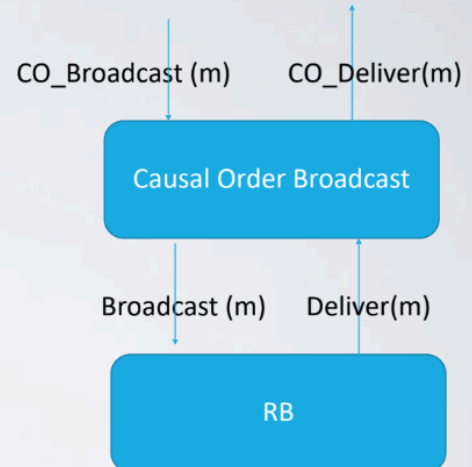
$pending := pending \cup \{(p, W, m)\};$

while exists $(p', W', m') \in pending$ such that $W' \leq V$ **do**

$pending := pending \setminus \{(p', W', m')\};$

$V[rank(p')] := V[rank(p')] + 1;$

trigger $\langle crb, Deliver \mid p', m' \rangle;$



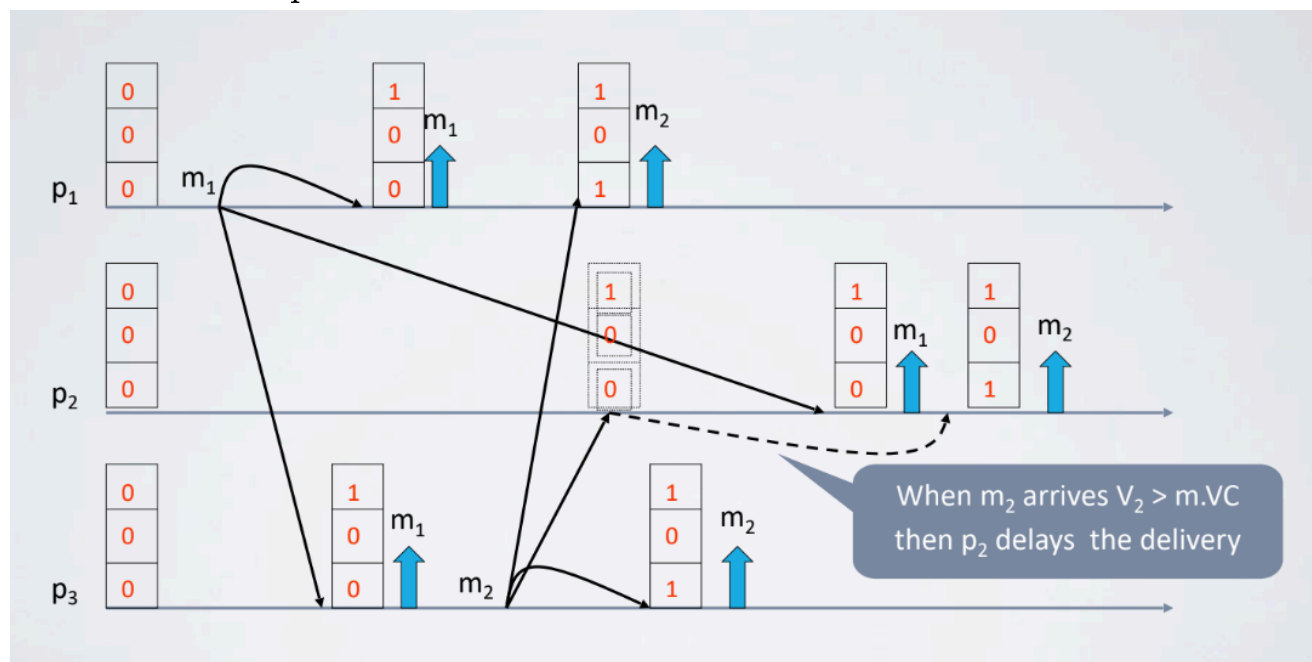
The vector clock used here is a bit different from the one we saw. Here, the value it's increased before sending the message, not after. Along side the vector clock, we also have a local sequence number. When we broadcast a message, we create a copy of V named W , and we update our vector clock to our sequence number. We update our vector clock then and broadcast the message.

When a process receives a message, we update the pending set, and then we check (similar to FIFO) if there is an entry with a vector clock smaller (and equal) than ours.

Remember, the value is updated after, so for the first message we have 0 as value, for the second 1.

With this algorithm, we cannot violate FIFO.

Let's see an example:



p_1 at first will have $(0, 0, 0)$. When p_3 receives the message, he increases the component of p_1 by 1. When p_3 sends m_2 , he updates its lsn , and sends $(1, 0, 0)$. p_1 will receive his own message some-when, and will update its component as well. When p_1 receives a message, he compares $(1, 0, 0)$ to the one received $(1, 0, 0)$, and because it's $\leq$ he can deliver it, increasing the counter for p_3 . When p_2 receives the message with $(1, 0, 0)$ he cannot deliver it yet, so he has to wait. So, when he finally receives m_1 , he delivers that and then m_2 .

Now, if we increase our component when we receive our message as broadcast, why should we increase also lsn ?

Causal broadcast is strong, but it cannot solve all problems.

In a system where we have money, causal broadcast is enough, but if we also have interests (for example) it's not enough.

If operations are concurrent, we may have a problem.

So, we take a look at total order broadcast (also known as atomic broadcast).

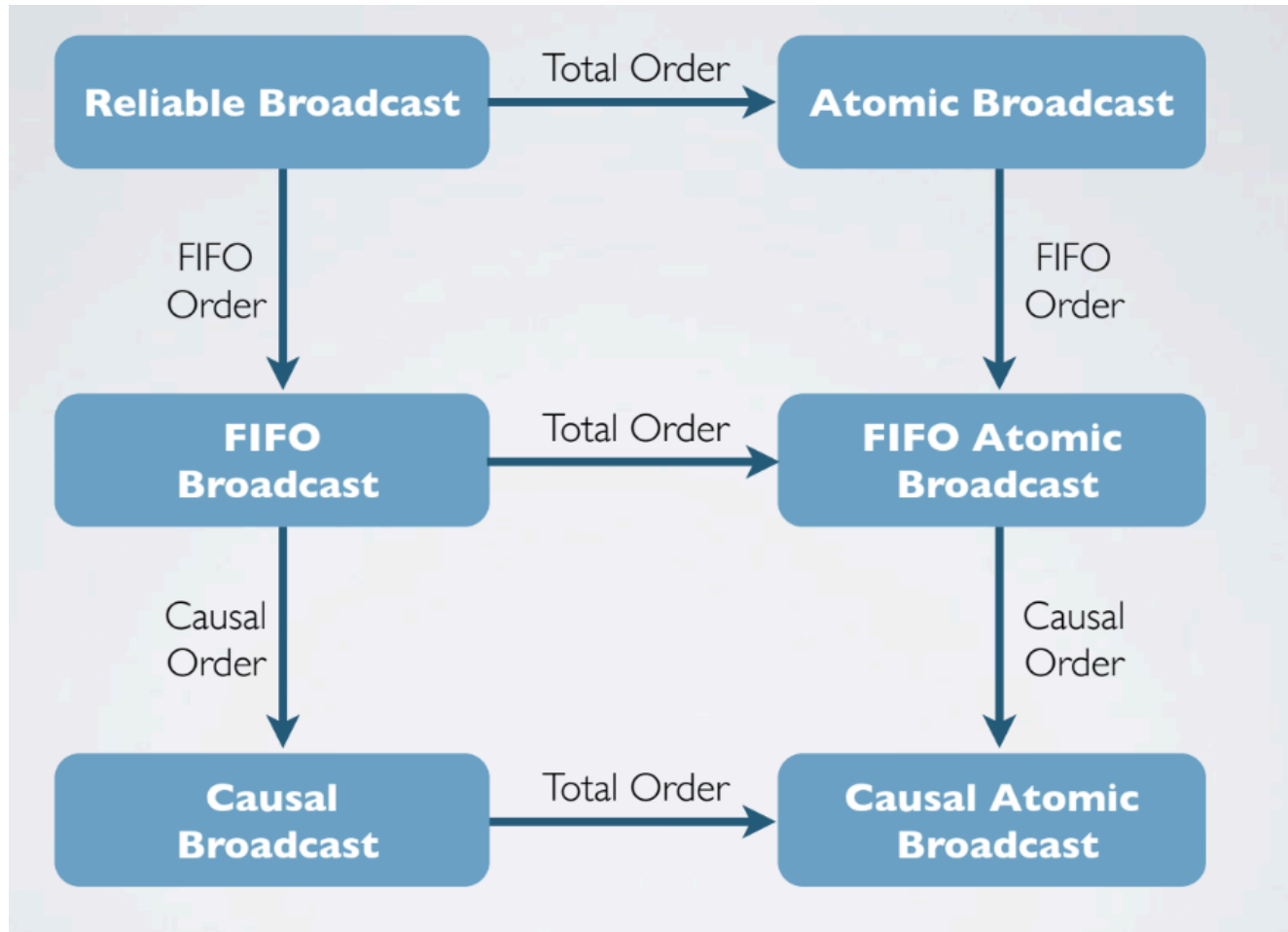
What are the properties? If there is a process p_1 that sees messages in the order of m_1, m_2, m_3 , then everyone will see the messages in the same order.

So, the total order broadcast orders the message that are concurrent.

There is a difference between FIFO and Causal Order, they are orthogonal.

In fact, m_2 could be delivered before m_1 , but then every process should deliver m_2 before m_1 , even if m_1 was sent before m_2 .

So, what is the relationship between broadcasts?



For example, the difference between causal and causal atomic is that concurrent messages could be ordered in different ways in the causal, while in causal atomic the concurrent are ordered in the same way for everyone.

We'll see total ordering later during the lectures, after seeing consensus.

Now, let's see an exercise.

Exercise 3.11: Compare the causal delivery property of Module 3.9 with the following property: “If a process delivers messages m_1 and m_2 , and $m_1 \rightarrow m_2$, then the process must deliver m_1 before m_2 .”

CRB5: Causal delivery: For any message m_1 that potentially caused a message m_2 , i.e., $m_1 \rightarrow m_2$, no process delivers m_2 unless it has already delivered m_1 .

Are these two properties the same or different? If they are the same, they allow the same set of executions, otherwise they don't. We call P_1 the first set, and P_2 the second. If they are different, then $P_1 \cap P_2 = \emptyset$, or we may have $P_1 \subset P_2$ or $P_2 \subset P_1$ or $P_1 = P_2$.

For pr_1 , for example, we have that

if $(D(m_1), D(m_2)) \wedge m_1 \rightarrow m_2 \implies O(m_1, m_2)$ (O is the order)

For pr_2 :

if $m_1 \rightarrow m_2 \implies O(m_1, m_2)$.

So the implication is the same, but the predicate is different.

So what is the answer? It's $P_1 \subset P_2$. There could be an execution that enters in the if of pr_2 and not in the if of pr_1 . So, there are executions that can enter in P_2 but not in P_1 .

Let's see an execution:

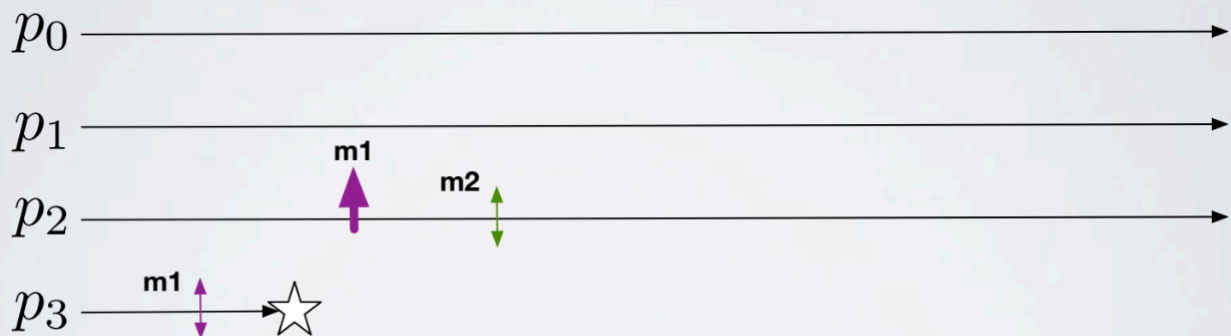
We have three messages, with relationship $m_0 \rightarrow m_1 \rightarrow m_2$.

There is process p_0 delivering m_0, m_1 and m_2 . p_2 , however, delivers m_0 and m_2 .

For p_2 we have that $m \rightarrow m'$ so the order is correct. However we are violating pr_1 , because it does not allow to skip m_1 .

Let's do another exercise:

Exercise 3.12: *Can we devise a best-effort broadcast algorithm that satisfies the causal delivery property without being a causal broadcast algorithm, i.e., without satisfying the agreement property of a reliable broadcast?*



So, can we create an algorithm that respects causal order, but has no agreement property (if a correct receives a message, then all the other corrects will)?

In the example, to have validity satisfied, we should have m_2 delivered by everyone. But can I do it without delivering also m_1 ? (without violating causal)
No, because the past of p_2 contains also m_1 , so every other process receiving m_2 should wait for m_1 .

check this thing better

Registers

They are memory locations shared between processes. Registers are important because they are implemented in CPU.

We'll see how to implement them by using messages. Also, we'll have to implement

registers with different properties, with the differences of what I can read in a register if someone is writing on it at that moment.